

Análisis del Modelo de Datos y Generación de Datos Sintéticos para FleetLogix

Introducción

FleetLogix es una empresa de transporte y logística que opera una flota de 200 vehículos realizando entregas de última milla en 5 ciudades principales. Este documento presenta el análisis del modelo de datos relacional y la estrategia de generación de datos sintéticos para modernizar su infraestructura de datos.

Análisis del Modelo Proporcionado

Estructura de Tablas y Relaciones

El modelo de datos de FleetLogix consta de 6 tablas interrelacionadas:

vehicles (vehículos de la flota)

- PK: vehicle_id (SERIAL)
- Constraints:
 - license_plate VARCHAR(20) UNIQUE NOT NULL
 - status VARCHAR(20) DEFAULT 'active'
- Descripción: Almacena información de los vehículos de la flota

drivers (conductores)

- PK: driver_id (SERIAL)
- Constraints:
 - employee_code VARCHAR(20) UNIQUE NOT NULL
 - license_number VARCHAR(50) UNIQUE NOT NULL
 - status VARCHAR(20) DEFAULT 'active'
- Descripción: Gestiona la información del personal conductor

routes (rutas predefinidas)

- PK: route_id (SERIAL)
- Constraints: route_code VARCHAR(20) UNIQUE NOT NULL
- Descripción: Define las rutas entre ciudades con distancias y costos

trips (viajes realizados)

- PK: trip_id (SERIAL)
- FK:
 - vehicle_id → vehicles(vehicle_id)

- driver_id → drivers(driver_id)
- route_id → routes(route_id)
- Constraints: status VARCHAR(20) DEFAULT 'in_progress'
- Descripción: Registra los viajes realizados con sus métricas

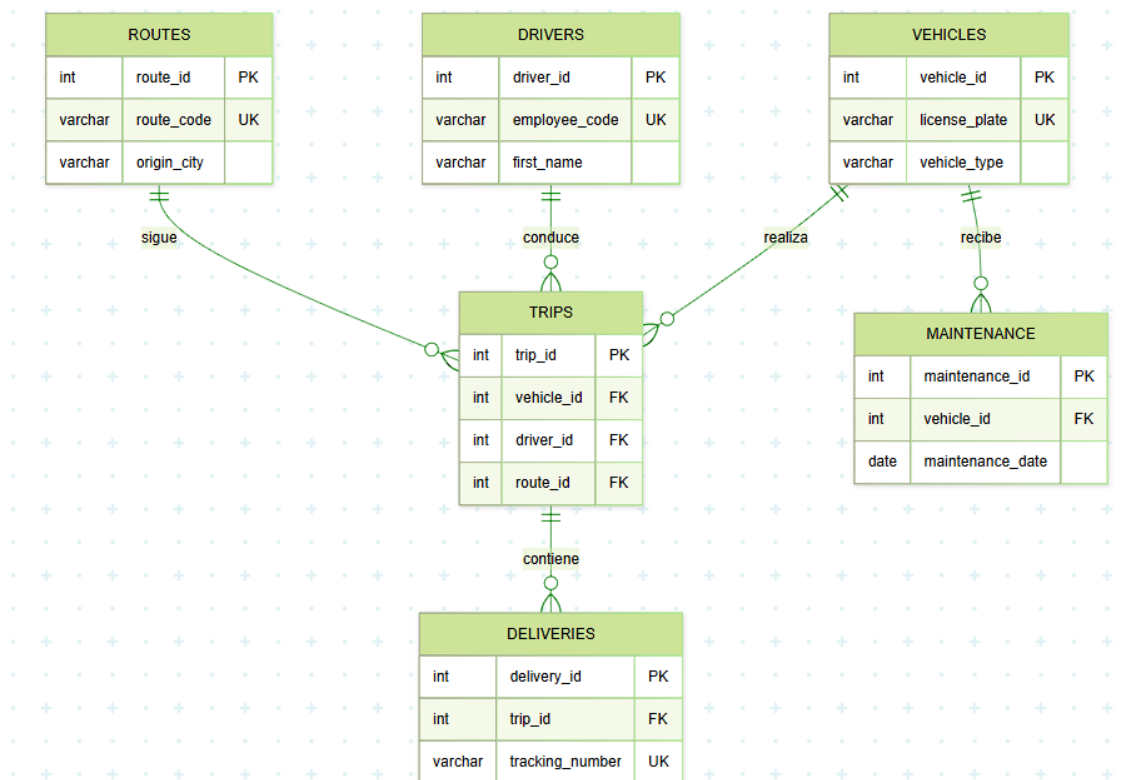
deliveries (entregas individuales)

- PK: delivery_id (SERIAL)
- FK: trip_id → trips(trip_id)
- Constraints:
 - tracking_number VARCHAR(50) UNIQUE NOT NULL
 - delivery_status VARCHAR(20) DEFAULT 'pending'
 - recipient_signature BOOLEAN DEFAULT FALSE
- Descripción: Detalla cada entrega realizada en los viajes

maintenance (mantenimiento de vehículos)

- PK: maintenance_id (SERIAL)
- FK: vehicle_id → vehicles(vehicle_id)
- Descripción: Registra el historial de mantenimiento de la flota

Diagrama ER



Índices Existentes

- CREATE INDEX idx_trips_departure ON trips(departure_datetime);
- CREATE INDEX idx_deliveries_status ON deliveries(delivery_status);
- CREATE INDEX idx_vehicles_status ON vehicles(status);

Estrategia de Generación de Datos Sintéticos

Metodología de Generación

Datos Maestros (650 registros)

- vehicles: 200 registros (4 tipos: Camión Grande, Camión Mediano, Van, Motocicleta)
- drivers: 400 registros (con licencias válidas y fechas coherentes)
- routes: 50 registros (conectando 5 ciudades principales)

Datos Transaccionales (504,400 registros)

- trips: 100,000 registros (2 años de operación histórica)
- deliveries: 400,000 registros (entre 2-6 entregas por viaje)
- maintenance: 5,000 registros (mantenimiento cada ~20 viajes)

Algoritmos Clave

* generate_trips(): en Python

```
def generate_trips_data(n_trips, vehicles_df, drivers_df, routes_df):  
    """
```

Genera viajes considerando:

- Distribución horaria realista
- Consistencia temporal (arrival > departure)
- Consumo de combustible basado en distancia
- Capacidad de carga coherente con el vehículo

```
    """
```

- get_hourly_distribution(): en Python

```
def get_hourly_distribution():  
    """
```

Distribución probabilística basada en patrones logísticos:

- 6-9 am: 8% (hora pico mañana)
- 9-12 am: 6% (mañana)
- 12-3 pm: 5% (medio día)
- 3-6 pm: 7% (tarde)
- 6-9 pm: 9% (hora pico tarde)
- 9 pm-6 am: 2% (noche/madrugada)

```
    """
```

Control de Calidad

Validaciones Implementadas

1. **Integridad Referencial:** Todas las claves foráneas existen en tablas padre

2. **Consistencia Temporal:** arrival_datetime > departure_datetime
3. **Unicidad:** Valores únicos respetados para license_plate, employee_code, etc.
4. **Distribución Realista:** Datos siguen patrones empresariales reales

Implementación Técnica

Tecnologías Utilizadas

- **Python 3.8+** con pandas para manipulación de datos
- **Faker** para generación de datos sintéticos realistas
- **Psycopg2** y **SQLAlchemy** para conexión a PostgreSQL
- **Jupyter Notebook** para desarrollo interactivo

Estructura del Script: en Python

1. **Preparación inicial:** Configuración → Parámetros → Herramientas → Conexión BD → Utilidades
2. **Generación de datos maestros:** Vehículos → Conductores → Rutas → Inserción Robusta → Población BD
3. **Generación e inserción de datos transaccionales:** Viajes → Entregas → Mantenimiento → Inserción Controlada
4. **Validación de calidad y breve análisis:** Completitud → Estadísticas → Visualización → Documentación

Parámetros Configurables: en Python

- ✓ NUM_VEHICLES = 200
- ✓ NUM_DRIVERS = 400
- ✓ NUM_ROUTES = 50
- ✓ NUM_TRIPS = 100000
- ✓ NUM_DELIVERIES = 400000
- ✓ NUM_MAINTENANCE = 5000

Conclusiones

El modelo de datos proporcionado es robusto y adecuado para las operaciones de FleetLogix. La generación de datos sintéticos ha permitido crear un dataset de más de 500,000 registros que mantiene todas las relaciones business y constraints técnicos, proporcionando una base sólida para el desarrollo del sistema de análisis de datos.